Laboratorijska vježba 3

FM-indeks: pretraživanje dok tipkate

Napredni algoritmi i strukture podataka

Opis vježbe

Na 3. laboratorijskoj vježbi zadatak je napraviti **dinamičko pretraživanje teksta**, pomoću **FM-indeksa** iz knjige. Korisnik upisuje slovo po slovo, a nakon svakog slova program kaže gdje se trenutni uzorak nalazi u tekstu. Prije vježbe trebali biste implementirati Burrows-Wheelerovu transformaciju (BWT), tablicu C, brojanje occ, pretraživanje unatrag i pronalazak pozicija pomoću spremljenih uzoraka (*sampling*). Možete krenuti od koda knjige i njezina repozitorija ili napisati svoj kod; struktura koda je slobodna.

Indeks ne morate komprimirati. Dovoljno je držati zadnji stupac BWT-a kao običan niz znakova. occ možete računati pomoću spremljenih brojača, kao u knjizi u poglavlju 3.7.2. Varijanta indeksa bez kompresije iz repozitorija knjige (`mbrcic/algodesign`) je `fm_index_plain.py`.

Opis zadatka: Program jednom izgradi indeks nad zadanim tekstom (mala slova `a-z` i znak `_` kao razmak između riječi). Nakon što izgradi indeks, program prima pritiske tipke. U Edgaru čemo simulirati pritiske tipkovnice naredbama. Prema tome, nije potrebno implementirati stvarno praćenje pritisaka tipkovnice. Za svako upisano ili izbrisano slovo ("pritisak tipkovnice"), program za trenutni uzorak daje interval redaka indeksa, broj pojavljivanja i početne pozicije u tekstu. Rezultati trebaju odgovarati onima u primjerima.

Točan zadatak te format ulaza i izlaza dobit ćete na samoj vježbi.

Primjer

U tablicama je ← brisanje zadnjeg slova.

Primjer 1. Tekst je `banana` (isti kao u knjizi, odjeljak 3.7.1), duljine $n = 6$.

tipka	uzorak	retci	broj	pozicije
`b`	`"b"`	$[4, 5)$	1	0
`a`	`"ba"`	$[1, 2)$	1	0
`n`	`"ban"`	$[5, 6)$	1	0
←	`"ba"`	$[1, 2)$	1	0
←	`"b"`	$[4, 5)$	1	0
←	`""`	$[0, 7)$	7	0, 1, . . . , 6
`a`	`"a"`	$[1, 4)$	3	1, 3, 5
`n`	`"an"`	$[5, 7)$	2	1, 3
`x`	`"anx"`	nema	0	nema
←	`"an"`	$[5, 7)$	2	1, 3
`a`	`"ana"`	$[2, 4)$	2	1, 3

Primjer 2. Tekst je `ana_voli_milovana`, duljine $n = 17$.

tipka	uzorak	retci	broj	pozicije
`a`	`"a"`	$[3, 7)$	4	0, 2, 14, 16
`n`	`"an"`	$[12, 14)$	2	0, 14
`a`	`"ana"`	$[4, 6)$	2	0, 14
`_`	`"ana_"`	$[1, 2)$	1	0
←	`"ana"`	$[4, 6)$	2	0, 14
←	`"an"`	$[12, 14)$	2	0, 14
←	`"a"`	$[3, 7)$	4	0, 2, 14, 16
←	`""`	$[0, 18)$	18	0, 1, . . . , 17
`l`	`"l"`	$[9, 11)$	2	6, 11
`x`	`"lx"`	nema	0	nema
←	`"l"`	$[9, 11)$	2	6, 11
`o`	`"lo"`	$[14, 15)$	1	11

Literatura

- 3.3 I remember how it starts: pretraživanje dok tipkate za početke riječi (trie) i 3.3.5, što početak ne može
- 3.6 Useful mess?: BWT, LF i dekodiranje (3.6.1–3.6.3)
- 3.7 By heart: FM-indeks, pretraživanje unatrag i brzo brojanje (3.7.1–3.7.3)

- 3.8.1 Where the match sits: pozicije pomoću uzoraka
- Kod knjige (repozitorij `mbrcic/algodesign`, mapa `python/algodesign/strings`): `fm_index_plain.py` (indeks bez kompresije), `count_index.py` (brojanje occ), `bwt.py`, `suffix_array.py`

Istražite sami

1. **BWT ručno.** Napišite BWT za `banana$` i rekonstruirajte tekst iz zadnjeg stupca kao u knjizi.
2. **Pretraživanje unatrag.** Ponovite primjer iz knjige: `ana` u `banana`. Za svako pročitano slovo c funkciju $occ(c, j)$ pozivate dvaput. Za koje j i zašto?
3. **Primjeri.** Reproducirajte svaki redak obiju tablica: interval, broj i pozicije.
4. **Provjera.** Na kratkim slučajnim tekstovima usporedite broj i pozicije pojavljivanja s običnim pretraživanjem teksta. Ispitajte i pojavljivanja na samom početku i na samom kraju teksta.

Za razmišljanje

- Zašto intervali redaka u primjerima nisu oni koje knjigin indeks daje za isti tekst?
- Koliko košta jedan pritisak tipke i o čemu ta cijena ovisi: o duljini uzorka, o duljini teksta? Koliko bi koštalo nakon svakog pritiska ponovno pretražiti cijeli uzorak, a koliko pretražiti cijeli tekst bez indeksa?
- Kada korisnik obriše zadnje slovo, kako dobiti interval prethodnog uzorka bez ponovnog pretraživanja? Vrijedi li isto rješenje ako prije brisanja uzorak nije postojao u indeksu? Npr. u tekstu `banana` uzorak `"ann"` ne postoji, a brisanjem zadnjeg slova dobiva se postojeći uzorak `"an"`.
- Gdje u poretku znakova moraju biti `$` i `_` da intervali ispadnu kao u primjerima? Zašto `$` mora biti jedinstven i manji od svih slova?
- Koje retke obuhvaća prazan uzorak i koliko se puta on pojavljuje?
- Koliko memorije troši pamćenje prethodnih stanja i kako to ovisi o duljini uzorka?
- Trie i Patricia stablo iz odjeljka 3.3 također podržavaju pretraživanje dok tipkate. Što smo FM-indeksom dobili, a što izgubili u odnosu na njih?

Kako teče vježba

Na vježbi dobivate točan zadatak i svoje pripremljeno rješenje prilagođavate njemu. Program čita sa standardnog ulaza, piše na standardni izlaz i izvodi se na skrivenim testovima, koji nakon svakog pritiska tipke uspoređuju ispis. Imate **2 sata** da program prođe sve testove i da započnete kviz. Kviz o svojem rješenju i gradivu otključava se **tek kada program prođe sve testove**, traje **10 minuta** i ima **samo jedan pokušaj**.

Dozvoljeni jezici

Jezik	Inačica i opcije prevođenja
Python	3.12.15
C	GCC 15.2.0, `-O2`
C++	GCC 15.2.0, `-O2`, `-std=c++23`
Rust	1.98.1, `--edition=2021`, `opt-level=3`
Java	OpenJDK 21.0.2